Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 5: Implementation of CI/CD Pipeline Using GitHub Actions and Jenkins

Submitted By:

Sumin GC

Roll no. 40

Submitted To:

Er. Rajendra Bahadur Thapa

Lab Report

Title

Implementation of Continuous Integration and Continuous Deployment (CI/CD) Using GitHub Actions and Jenkins

Objective

To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions and Jenkins for automating the software build, testing, and deployment processes, thereby improving development efficiency and software quality.

Tools Required

- Git
- GitHub
- GitHub Actions
- Jenkins
- Java or Node.js (Sample Project)
- Visual Studio Code

Theory

Continuous Integration (CI) is a software development practice in which code changes are automatically built and tested whenever they are committed to a shared repository. This process helps identify integration issues early, ensuring that newly added code does not introduce errors into the project.

Continuous Deployment (CD) extends Continuous Integration by automatically deploying successfully tested applications to a target environment with minimal or no manual intervention. Together, CI/CD enables faster software delivery, reduces manual effort, and improves application reliability.

GitHub Actions is GitHub's built-in automation platform that executes workflows defined in YAML files whenever specified events, such as code pushes or pull requests, occur. Jenkins is an open-source automation server that supports building, testing, and deploying applications through customizable pipelines, making it one of the most widely used CI/CD tools.

Procedure

1. Create a sample project and upload it to a GitHub repository.
2. Create a GitHub Actions workflow file inside the .github/workflows directory.
3. Define the workflow to automatically execute when code is pushed to the main branch.
4. Commit and push the workflow file to GitHub to trigger the CI pipeline.
5. Install and configure Jenkins on the local machine or server.
6. Connect Jenkins to the GitHub repository using the repository URL.
7. Create a Jenkins pipeline and define the build, test, and deployment stages.
8. Execute the Jenkins pipeline and verify that each stage completes successfully.
9. Review the workflow logs and pipeline results to confirm successful automation.

GitHub Actions Workflow

```
name: CI Pipeline

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
     -uses: actions/checkout@v4

     -name: Set up Java
       uses: actions/setup-java@v4
       with:
         distribution: temurin
         java-version: '17'

     -name: Build Project
       run: echo "Build Successful"
```

```
JenkinsPipeline Example
pipeline {
    agentany

    stages {

        stage('Build') {
            steps {
                echo 'Building Project...'
            }
        }

        stage('Test') {
            steps {
                echo 'Running Tests...'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying Application...'
            }
        }

    }
}
```

Output

- The sample project was successfully uploaded to GitHub.
- A GitHub Actions workflow was created and executed automatically after code was pushed to the repository.
- The project build completed successfully through GitHub Actions.
- Jenkins was installed, configured, and connected to the GitHub repository.
- A Jenkins pipeline was successfully created with Build, Test, and Deploy stages.
- The pipeline executed successfully, completing all stages without errors. •

Workflow logs and pipeline execution results confirmed successful automation of the software development process.

Result

The experiment successfully implemented a Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions and Jenkins. The automated workflow performed project building, testing, and deployment, demonstrating an efficient and reliable software development process.

Conclusion

This experiment provided practical experience in implementing CI/CD pipelines using GitHub Actions and Jenkins. The automated workflows simplified software development by reducing manual intervention, ensuring continuous code integration, and streamlining application deployment. Implementing CI/CD practices improves software quality, accelerates development cycles, enhances collaboration among development teams, and enables faster and more reliable software delivery.